

Trabalho P1 – Biblioteca de Grafos em C++

Teoria dos Grafos – COS 242

Dupla: Carlos Bruno – Arthur Cury

22 de setembro de 2026

Código-fonte: <https://github.com/Ludwigo0/TrabalhoGrafos>

1 Projeto e implementação: o que fizemos e por quê

Neste trabalho, buscamos desenvolver uma biblioteca de grafos que pudesse ser utilizada em outros programas, e não apenas um executável feito para resolver um caso específico. Por esse motivo, dividimos o projeto em duas partes: a biblioteca (**libgrafo**), que concentra as representações e os algoritmos, e um programa-cliente (**grafo**), responsável por ler os argumentos, chamar as funções da biblioteca, medir os tempos e gravar os resultados.

Essa separação facilita a reutilização do código. Um novo programa pode incluir os cabeçalhos da biblioteca e utilizá-la sem precisar copiar a implementação dos algoritmos. Assim, a estrutura do projeto atende melhor ao conceito de biblioteca reutilizável apresentado no enunciado.

1.1 Guardar o grafo: lista CSR e matriz de bits

Existem duas formas clássicas de armazenar um grafo. Na **lista de adjacência**, cada vértice guarda apenas os seus vizinhos, fazendo com que o espaço utilizado dependa principalmente do número de arestas. Já na **matriz de adjacência**, é mantida uma tabela com uma posição para cada par de vértices. Nesse caso, o consumo de memória depende de n^2 , mesmo quando o grafo possui poucas arestas. Essa diferença fica evidente nos grafos utilizados. Para o grafo 1, com 10 mil vértices, uma matriz ainda pode ser considerada. Para o grafo 6, com aproximadamente 4,8 milhões de vértices, a quantidade de posições seria impraticável. Por isso, um dos objetivos dos experimentos foi observar na prática como a escolha da representação influencia o consumo de memória e o tempo de execução.

Para a lista de adjacência, evitamos utilizar diretamente um `vector<vector<int>>`. Embora essa estrutura seja simples, cada vetor possui um custo de administração e também pode causar fragmentação de memória. Em grafos muito grandes, esse custo adicional se torna relevante.

Em seu lugar, utilizamos o formato **CSR** (*Compressed Sparse Row*). Ele usa dois vetores contíguos: `offsets[n+1]`, que indica onde começa e termina a lista de cada vértice, e `edges[2m]`, que armazena os vizinhos. No grafo não direcionado, cada aresta aparece nas listas dos dois extremos. Essa organização reduz o desperdício de memória e permite percorrer os vizinhos de forma sequencial, aproveitando melhor a localidade de cache.

Como o grafo é carregado uma vez e depois apenas consultado, o CSR se mostrou uma escolha adequada para este trabalho, mesmo não sendo a melhor opção para grafos que precisam sofrer muitas inserções ou remoções.

Na matriz, cada posição poderia ser armazenada em um byte, mas optamos por guardar **um bit por posição**, agrupando os bits em palavras de 64 bits. Com isso, o consumo cai para aproximadamente $n^2/8$ bytes. No grafo 2, essa escolha reduz a memória estimada de cerca de 2,3 GB para aproximadamente 297 MB. O custo adicional aparece no tempo de processamento: para acessar uma posição, precisamos calcular a palavra e a máscara correspondentes. Durante a BFS, as linhas da matriz precisam ser percorridas palavra por palavra. Também foi necessário utilizar tipos de 64 bits no cálculo das posições, pois n^2 ultrapassa o limite de um inteiro de 32 bits já no grafo 2. Para evitar uma tentativa de alocação muito grande, a matriz possui uma trava de segurança para $n > 60000$.

(aproximadamente 450 MB): acima disso o construtor recusa a operação e informa a memória que seria necessária. O resultado é registrado como *OOM teórico*, e não como um crash inesperado.

As duas representações implementam a mesma interface abstrata, chamada **Graph**. Assim, os algoritmos recebem uma referência para essa interface e não precisam conhecer os detalhes internos da estrutura utilizada. Para trocar de CSR para matriz, basta alterar a opção `--rep lista|matriz` na linha de comando. Essa decisão simplifica os testes e permite comparar as representações utilizando os mesmos algoritmos.

1.2 Explorar o grafo: BFS e DFS

A **BFS** (busca em largura) explora o grafo por camadas. Primeiro, visita os vizinhos da origem; depois, os vizinhos desses vértices, utilizando uma fila. Por causa dessa ordem, o nível atribuído a cada vértice representa a menor distância até a origem. Por isso, a BFS pode ser usada como base para calcular distâncias, componentes e diâmetros.

A **DFS** (busca em profundidade) segue uma ideia diferente: avança por um caminho até não encontrar mais vértices novos e depois retorna para continuar a busca. O nível produzido pela DFS indica a profundidade de descoberta, e não a menor distância. Isso fica claro nas tabelas: a DFS alcança profundidades de milhões, enquanto os níveis da BFS permanecem pequenos.

Implementamos as duas buscas de forma **iterativa**. Essa decisão evita que uma DFS recursiva estoure a pilha de chamadas nos grafos maiores. Na representação por matriz, a busca percorre cada linha por palavras de 64 bits, utilizando `forEachNeighbor`. Dessa forma, o custo fica em torno de $O(n^2)$ por busca. Uma implementação que repetisse a varredura da linha para cada vizinho seria muito mais lenta; no grafo 2, a varredura otimizada reduziu a penalidade observada para cerca de 11 vezes na BFS (cerca de 7 vezes na DFS) em relação ao CSR.

Os despachantes, como `bfsAuto`, escolhem automaticamente a implementação mais adequada sem expor os detalhes da representação ao chamador. Os níveis da BFS foram iguais nas duas estruturas, conforme conferido com `diff`. Os pais, por outro lado, podem ser diferentes, pois uma mesma busca pode possuir mais de um predecessor válido e a escolha depende da ordem dos vizinhos. A saída segue o formato pedido: para cada vértice, são registrados o pai e o nível; a raiz não possui pai e os vértices inalcançáveis recebem nível -1 .

1.3 Distância e diâmetro: o exato e o possível

A distância entre dois vértices é calculada por uma BFS com parada antecipada: assim que o destino é alcançado, a busca termina. O diâmetro exato, definido como a maior distância entre dois vértices, exigiria uma BFS a partir de cada vértice, com custo $O(n \cdot (n + m))$. Considerando os tempos medidos, somente no grafo 6 seriam necessárias aproximadamente 4,8 milhões de BFSs, o que ultrapassaria 30 dias de execução.

Por esse motivo, utilizamos o método aproximativo *double-sweep*. Primeiro, executamos uma BFS a partir de um vértice qualquer e escolhemos o vértice mais distante, chamado *a*. Em seguida, executamos uma nova BFS a partir de *a* e escolhemos o vértice mais distante, chamado *b*. O valor retornado é $dist(a, b)$. A ideia é que a primeira busca tende a encontrar uma região próxima de uma das extremidades do diâmetro.

O método utiliza apenas duas BFSs, em vez de n , e por isso continua viável nos grafos grandes. Entretanto, ele não garante o diâmetro exato: o valor encontrado é sempre um limite inferior do diâmetro real. Para grafos desconexos, consideramos a maior componente, pois o diâmetro de um grafo desconexo não possui um valor finito para pares de vértices em componentes diferentes.

1.4 Componentes, estatísticas, entrada e medição

Para encontrar as componentes conexas, percorremos os vértices e iniciamos uma BFS sempre que encontramos um vértice ainda não rotulado. Cada busca marca exatamente uma componente, e as componentes são apresentadas em ordem decrescente de tamanho. Nas estatísticas de grau, utilizamos `nth_element` para encontrar a mediana sem precisar ordenar os aproximadamente 4,8 milhões de valores do maior grafo.

A leitura também exigiu uma decisão de implementação. O grafo 6 possui cerca de 722 MB e 93 milhões de inteiros; por isso, uma leitura ingênua com `cin`>> seria muito demorada. Implementamos um scanner que lê blocos de 1 MB com `fread` e converte os números manualmente.

O CSR é construído em **duas passadas** pelo arquivo: primeiro contamos os graus, depois calculamos os offsets acumulados e, por fim, preenchemos o vetor de arestas. Essa abordagem evita manter uma lista temporária de aproximadamente 370 MB no grafo 6. Os tempos dos algoritmos são medidos com `chrono`, sem incluir a leitura e a escrita dos arquivos, conforme solicitado no enunciado. A memória de pico é obtida por meio do campo `VmHWM` de `/proc/self/status`, o que torna a medição mais reprodutível do que uma observação manual do monitor do sistema.

Como reproduzir. A compilação pode ser feita com `cmake -S . -B build && cmake --build build -j`, utilizando C++17. Um exemplo de execução é:

```
./build/grafos grafo_1.txt --rep lista --stats --components \
--bfs 1 --diameter-approx 1
```

O script `./scripts/run_estudos.sh` executa todos os experimentos e gera o arquivo CSV utilizado nas tabelas. O programa `./build/test_grafos` verifica o exemplo da Fig. 1 nas duas representações, a equivalência das vias rápidas, a leitura de um grafo desconexo e a trava da matriz. Na compilação em modo Debug, os `asserts` também são ativados. Os vértices informados pela linha de comando seguem a numeração do arquivo, de 1 a n .

2 Estudos de caso – tabelas

Tabela 1: Características (lista CSR) e diâmetro aproximado.

Grafo	n	m	gmin	gmax	gméd	gmed	Comp. (maior/menor)	Diâm.
1	10 000	109 977	8	43	21,99	22	1 (10 000/10 000)	4
2	49 948	1 299 794	1	112	52,04	55	10 (25 000/48)	5
3	375 000	765 618	1	15	4,08	4	2 (250 000/125 000)	22
4	375 000	8 187 427	9	89	43,67	47	2 (250 000/125 000)	4
5	4 843 750	13 168 928	1	22	5,44	5	5 (2 500 000/156 250)	12
6	4 843 750	46 469 670	2	56	19,19	20	5 (2 500 000/156 250)	7

Q1 – memória (MB de pico). A lista CSR utilizou 6, 15, 15, 72, 161 e 415 MB nos grafos 1 a 6, respectivamente. A matriz de bits utilizou 16,8 MB no grafo 1 e 302 MB no grafo 2. Para os demais grafos, a matriz não é viável: seriam necessários aproximadamente 17,6 GB nos grafos 3 e 4, e 2,93 TB nos grafos 5 e 6. Esses valores ultrapassam a memória de 15 GiB disponível na máquina utilizada.

Q2/Q3 – tempo médio de 100 buscas (ms).

Grafo	BFS lista	BFS matriz	DFS lista	DFS matriz
1	0,8	4,9 ($\sim 6\times$)	1,0	6,3
2	2,6	28,5 ($\sim 11\times$)	4,4	32,1
3	10,5	– (OOM)	18,5	–
4	106	–	80	–
5	863	–	1366	–
6	606	–	905	–

Unidade: todos os valores estão em milissegundos por busca. No grafo 3, os valores de 10,5 ms para BFS e 18,5 ms para DFS correspondem à média de três rodadas, cada uma com 100 origens. Fizemos isso porque a carga da máquina produziu variações consideráveis entre as rodadas; a variação observada nesse grafo foi de 9 a 37 ms, com valor típico próximo de 15 ms.

Na lista CSR, uma busca custa $O(n+m)$. Na matriz, a implementação precisa percorrer as palavras da matriz e pode chegar a $O(n^2)$ por busca. No grafo 2, isso significa examinar uma quantidade muito maior de posições do que na lista. Para reduzir esse custo, utilizamos uma varredura por linha com

operações de *popcount*. Sem essa otimização, a mesma linha seria percorrida repetidas vezes e a penalidade ultrapassaria $400\times$.

Q4 – pai (nível) dos vértices 10, 20, 30; “–” = inalcançável.

Grafo	BFS origem 1			BFS origem 2			BFS origem 3		
	10	20	30	10	20	30	10	20	30
1	2042(3)	8382(3)	8979(4)	8935(2)	7541(4)	8660(4)	7685(3)	9429(4)	5783(3)
2	–	–	–	31751(3)	–	–	–	46738(3)	18855(3)
3	–	–	141597(14)	158403(8)	75471(9)	–	158403(10)	319691(7)	–
4	194465(4)	274921(4)	136244(3)	–	–	–	–	–	–
5	1888350(8)	–	2502539(9)	–	–	–	3696745(9)	–	191713(9)
6	–	–	–	1677854(4)	4095628(5)	4231681(5)	–	–	–

Grafo	DFS origem 1			DFS origem 2			DFS origem 3		
	10	20	30	10	20	30	10	20	30
1	3075(9k)	3575(9k)	6071(5k)	6552(5k)	9429(7k)	6071(7k)	2344(9k)	8781(9k)	6071(8k)
2	–	–	–	18k(16k)	–	–	–	887(8k)	33k(3k)
3	–	–	142k(69k)	19k(152k)	142k(111k)	–	107k(67k)	75k(171k)	–
4	272k(160k)	371k(223k)	130k(189k)	–	–	–	–	–	–
5	906k(1,1M)	–	2,5M(1,8M)	–	–	–	906k(1,8M)	–	2,5M(2,0M)
6	–	–	–	4,5M(515k)	624k(2,1M)	2,9M(1,8M)	–	–	–

Os resultados deixam clara a diferença entre as buscas: a DFS pode atingir profundidades de milhões, enquanto a BFS registra a menor distância em níveis pequenos. Nas representações **CSRGraph** e **BitMatrixGraph**, os níveis da BFS foram idênticos, conforme verificado com **diff**. Os pais podem ser diferentes quando existem vários predecessores possíveis, pois a ordem dos vizinhos determina qual deles é escolhido.

Q5 – distâncias. Na ordem dos grafos 1 a 6, as distâncias para os pares (10,20), (10,30) e (20,30) foram, respectivamente, (3, –1, 9, 4, –1, 5), (3, –1, –1, 3, 9, 5) e (4, 3, –1, 4, –1, 5). O valor –1 indica que os vértices pertencem a componentes diferentes. No grafo 2, o vértice 10 está na primeira componente, enquanto 20 e 30 estão na segunda.

Q6 – componentes. O grafo 1 é conexo. O grafo 2 possui 10 componentes, os grafos 3 e 4 possuem 2 componentes, com tamanhos 250 mil e 125 mil, e os grafos 5 e 6 possuem 5 componentes. Nestes dois últimos, a maior componente tem 2,5 milhões de vértices e a menor tem 156.250. As listas completas de membros estão no arquivo **components.txt**, em ordem decrescente de tamanho.

Q7 – diâmetro. Os valores aproximados obtidos pelo *double-sweep*, com duas BFSs, foram 4, 5, 22, 4, 12 e 7, respectivamente. O cálculo exato é inviável para os grafos grandes: no grafo 6, seriam necessárias 4,8 milhões de BFSs, o que levaria mais de 30 dias com os tempos observados.

No grafo 1, foi possível calcular o diâmetro exato, cujo valor foi 5. O processo levou cerca de 6s com a lista e 44s com a matriz. Assim, o resultado aproximado 4 foi confirmado como um limite inferior do valor real. No exemplo da Fig. 1, o valor exato e o aproximado também coincidiram em 2.

3 Análise: o que os números nos contaram

Os resultados confirmam a expectativa apresentada no início do relatório. Para os grafos testados, a lista CSR foi a alternativa mais adequada, principalmente quando o número de vértices cresceu. A matriz de bits ainda coube no grafo 2, mas apresentou tempo aproximadamente 11 vezes maior na BFS (cerca de 7 vezes na DFS). A partir do grafo 3, ela deixou de ser uma alternativa prática tanto pelo consumo de memória quanto pelo custo quadrático das buscas.

Essa conclusão deve ser entendida dentro do contexto dos experimentos realizados: os grafos são esparsos e os algoritmos avaliados são principalmente BFS e DFS. Em grafos densos ou em aplicações que realizem muitas consultas diretas de adjacência, a matriz pode ter outras vantagens. Mesmo assim, para o conjunto analisado, o CSR apresentou a melhor relação entre memória utilizada e tempo de execução.

Também identificamos algumas arestas repetidas nos grafos 1 e 2, com os vértices invertidos. A lista CSR mantém essas repetições, enquanto a matriz representa apenas a existência de cada vizinhança.

Isso explica as pequenas diferenças nas estatísticas de grau médio, de 0,047% no grafo 1 e 0,078% no grafo 2. Apesar disso, os níveis, as distâncias e as componentes encontradas foram os mesmos.

Por fim, os grafos 3 a 6 possuem componentes desconexas com tamanhos bem definidos. Essa característica explica as distâncias iguais a -1 e também ajuda a manter o custo do *double-sweep* controlado: cada BFS percorre somente a componente em que a busca foi iniciada. No grafo 6, por exemplo, duas BFSs ainda foram executadas em aproximadamente 0,5 a 1,5s, enquanto o cálculo exato exigiria uma quantidade de buscas impraticável.

De modo geral, o trabalho mostra que a escolha da estrutura de dados não é apenas uma decisão de implementação. Ela influencia diretamente a escalabilidade do programa e determina quais algoritmos podem ser executados em grafos grandes.